

Ping Pong Tanks

Printable work pack · game project

Ping Pong Tanks: your own GDD built in Unity up to the MVP — bouncing shells, a DFS maze and power-ups

Weeks 36–49 · autumn break in week 42 · handover Fri 4 Dec 2026

This pack is a schedule and a checklist for situations where the site is not open. The project journal is written on the site and pushed to the repository's project-docs folder. A tick in this booklet is not a submission — the work always lives in the Git repository.

The repository is public: no personal data, no school identifiers and no other people's names go into Git.

The GDD is your own — the implementation is judged against it.

Schedule

Week	Dates	Topic of the week	Phase
36	31 Aug – 4 Sep	GDD into tasks, project set up	A
37	7–11 Sep	Tanks move — two players, one keyboard	A
38	14–18 Sep	The bouncing shell	A
39	21–25 Sep	Rounds, scoring and the five-second rule	A
40	28 Sep – 2 Oct	DFS maze and data out of the code	B
41	5–9 Oct	Power-ups and the HUD	B
43	19–23 Oct	Client review and prioritisation	B
44	26–30 Oct	Feedback change and the chosen extra feature	C
45	2–6 Nov	Testing week and pair debugging	C
46	9–13 Nov	Menus and settings	C
47	16–20 Nov	RC, external testing and security	D
48	23–27 Nov	Release v1.0 (the MVP)	D
49	30 Nov – 4 Dec	Handover week	D

Handover no later than Fri 4 Dec 2026. The site has the detailed task instructions, the implementation help and the project journal.

Phase A — Core game: GDD into tasks, and a playable round

Week 36 · 31 Aug – 4 Sep — GDD into tasks, project set up

The GDD is split into an issue backlog, P0 is scoped with the client, and an empty Unity project compiles in a public repository.

- ☐ Read the brief and your own GDD side by side and write a question list for the client.
- ☐ Scope P0 with your instructor and split it into issues, each with a done-when condition.
- ☐ Set up the Unity project with the agreed version and a public GitHub repository — run the privacy check.
- ☐ Name the game team, hold the first weekly meeting and put the implementation plan into project-docs.

Done when: An outsider can see from the repository what is being built and in which order: the README describes the game in one paragraph, the milestone "MVP 4 Dec" holds the P0 issues, and the project compiles as an empty scene.

In the Git repository before you tick the box: In the repo: a public repository whose README, P0 milestone and first commit say what is being built — plus the question list with answers and the meeting note in project-docs.

Week 37 · 7–11 Sep — Tanks move — two players, one keyboard

Two tanks move on one keyboard in a test arena and stop at walls. The game is playable for the first time.

- ☐ Build the tank prefab (square body and round turret) with a Rigidbody2D and colliders.
- ☐ Implement two-player movement on one keyboard: WASD and the arrow keys.
- ☐ Record tests T01–T02 with their expectations before running them, and run them — plus the weekly meeting note.

Done when: Two players can move at the same time on one keyboard without the keys interfering with each other, and the tanks stop at a wall without jittering.

In the Git repository before you tick the box: In the repo: a playable build with two tanks moving, tests T01–T02 with results in tests.md, and the movement commits.

Week 38 · 14–18 Sep — The bouncing shell

The tanks shoot: a shell flies, ricochets off walls indefinitely and disappears after about 10 seconds. You can die to your own shell — the core tension of the game exists.

- ☐ Build the Projectile prefab with a physics material: Bounciness 1, Friction 0.
- ☐ Implement shooting: one shell per player on the field, slightly faster than the tank.
- ☐ Implement a ~10 s despawn and hits — including on your own tank (friendly fire).
- ☐ Record tests T03–T05 before running them: direct hit, ricochet chain, despawn time.

Done when: The shell bounces off walls without losing speed, disappears in about 10 seconds, kills its own shooter too, and a new shell can only be fired once the previous one is gone.

In the Git repository before you tick the box: In the repo: a video or image sequence of the ricocheting shell, tests T03–T05 with results, and the Projectile prefab commit.

Week 39 · 21–25 Sep — Rounds, scoring and the five-second rule

A hit destroys a tank, the round ends, a point is recorded and a new round starts. The GDD's five-second rule works: if both tanks die within five seconds, no point is awarded.

- ☐ Implement tank destruction and round cleanup: shells removed, input cut from the dead tank.
- ☐ Implement scoring, the GDD's five-second rule and the automatic round restart.
- ☐ Record tests T06–T08 before running them and play with the game team — note the round rhythm.

Done when: A whole round works without touching the editor: destruction, a point to the right player (or to neither, under the 5 s rule), and an automatic new round.

In the Git repository before you tick the box: In the repo: a whole round without the editor: scoring, the 5 s rule tested (T06–T08) and the playtest note week39.md.

Phase B — Arena and powers: maze, power-ups and the review

Week 40 · 28 Sep – 2 Oct — DFS maze and data out of the code

The arena is generated with a DFS algorithm: a 10×10 maze where every tile is reachable, and the tanks spawn fairly, at least 3 tiles apart. Arena data comes from outside the code.

- ☐ Write the data store comparison (ScriptableObject / JSON / PlayerPrefs) before implementing anything.
- ☐ Implement the DFS maze generator as its own class — draw the algorithm on paper first.
- ☐ Build the arena from data and implement spawn validation: starting positions at least 3 tiles apart.
- ☐ Record tests T09–T10 before running them: traversability and spawn distance.

Done when: Every generated 10×10 maze is fully traversable (DFS guarantees it), the tanks start at least 3 tiles apart, and the arena size is read from data — you can change the size without touching code.

In the Git repository before you tick the box: In the repo: the written data store comparison, the paper drawing of DFS, generated mazes and tests T09–T10.

Week 41 · 5–9 Oct — Power-ups and the HUD

Power-ups appear on the field roughly every 2 seconds in the centre of a tile, and speed boost, slowness and shield affect play. The tank and the HUD both show which power is active.

- ☐ Define the power-ups as data: SpeedBoost, Slowness and Shield assets with their durations.
- ☐ Implement spawning (~2 s interval, at tile centres), pickup and the one-power-at-a-time rule.
- ☐ Show the active powers in the HUD and on the tank, and record tests T11–T12 before running them.

Done when: Three powers work with their durations, a player can only hold one at a time (a new one is not picked up before the old one is spent — GDD), and a spectator can see both players' state from the HUD.

In the Git repository before you tick the box: In the repo: three power-up assets as data, a HUD screenshot and tests T11–T12 with results.

Week 43 · 19–23 Oct — Client review and prioritisation

The client has seen the game, the feedback is recorded in the client's own words, and the game team has made a prioritisation decision: which GDD extras fit into the remaining weeks. No new code is written this week.

- ☐ Prepare a 10-minute demo of the playable build and run the review for the client.
- ☐ Write the review note in two parts: the client's own words and your interpretation, kept separate.
- ☐ Make the prioritisation decision with the game team and update the implementation plan and the backlog.

Done when: The note holds the client's own words, your interpretation separately, and a named decision: what gets built in week 44, what moves to P1/P2. The instructor has signed off on the decision.

In the Git repository before you tick the box: In the repo: the review note in the client's words, the recorded prioritisation decision and the updated implementation plan.

Phase C — Finishing: feedback change, testing and menus

Week 44 · 26–30 Oct — Feedback change and the chosen extra feature

The change derived from the client's feedback is made on its own branch and merged through a pull request, and the feature chosen in the review is playable. The game team has played the version and assessed it together.

- ☐ Make the change derived from the feedback on its own branch and open a pull request that references the note.
- ☐ Implement the feature chosen in the review, on top of the week 38 and 41 systems.
- ☐ Run a game team playtest and record the shared assessment in a note.
- ☐ Record tests T13–T14 before running them and merge the PR after review.

Done when: Main holds a merged PR whose description makes clear which feedback it answers; the feature works; the playtest note holds at least three shared observations and a shared assessment.

In the Git repository before you tick the box: In the repo: a merged PR that references the feedback, the playable feature, the playtest note and tests T13–T14.

Week 45 · 2–6 Nov — Testing week and pair debugging

The test matrix has been run: at least 12 tests with results, three complete debugging chains verified, and one bug solved in a debugging pair with a team member. The game gets no new features — it becomes reliable.

- ☐ Complete the test matrix from the GDD's testing list: normal, boundary and error cases in balance.
- ☐ Run every test — the old ones again too — and record the result of every row.
- ☐ Solve one bug with a named debugging pair and record both sets of observations.
- ☐ Finish three debugging chains with all six parts.

Done when: Every row of the matrix has an expectation recorded before the run and the result of the run; the three chains hold an observation, reproduction steps, the cause, the fix commit, a retest and a regression test; the pair note holds both names and both sets of observations.

In the Git repository before you tick the box: In the repo: the run test matrix (≥12 rows), three complete debugging chains and the pair debugging note.

Week 46 · 9–13 Nov — Menus and settings

The game has the menu path the GDD describes: a main menu (Play / Options / Leave), settings and a pause menu. Settings persist between sessions, and the players cannot pick the same colour.

- ☐ Build the main menu and the pause menu per the GDD's UI chapter: Play, Options, Leave and resume/restart/quit.
- ☐ Implement the settings and save them to PlayerPrefs — block duplicate player colours.
- ☐ Record tests T15–T16 before running them: settings persistence and the duplicate colour block.

Done when: The game can be started, the settings changed and a match paused without touching the editor; the settings are still there on the next launch; the same player colour cannot be picked twice.

In the Git repository before you tick the box: In the repo: the playable menu path, PlayerPrefs settings persistence (T15) and the duplicate colour block (T16).

Phase D — Release and handover

Week 47 · 16–20 Nov — RC, external testing and security

A release candidate exists, and an outside tester has got the game running on another machine from the written instructions alone. The security of your own repository has been reviewed before release.

- ☐ Make the release candidate v1.0-rc1 and test it yourself in a clean folder.
- ☐ Write the installation and gameplay guide: download, launch, controls, goal.
- ☐ Arrange an outside tester on another machine with no spoken help, and record the findings.
- ☐ Run a security review of your own repository with concrete file paths and commits.

Done when: The tester got all the way through a match without spoken help; every point where they hesitated is recorded as a fix list for the guide; the security review names the files and commits it checked. After the RC, only blocking bugs get fixed.

In the Git repository before you tick the box: In the repo: the RC build, the written guide, the tester's findings in their own words and the security review with references.

Week 48 · 23–27 Nov — Release v1.0 (the MVP)

Ping Pong Tanks v1.0 (the MVP) is released: a GitHub Release on the agreed channel, a README that says everything needed, and a release tested in a clean environment. The content is frozen.

- ☐ Fix only the RC's blocking bugs and move the tester's fix list into the guide.
- ☐ Publish GitHub Release v1.0 with the build files and a scope description.
- ☐ Test the release in a clean environment, finish the README and LICENSE, and freeze the content.

Done when: An outsider can download the release and play a match on a machine that has never had the project on it; the README matches the GDD's controls and rules; the release page states what the MVP contains and what was deliberately left out (P1/P2).

In the Git repository before you tick the box: In the repo: GitHub Release v1.0, the clean-environment test report and the final README + LICENSE.

Week 49 · 30 Nov – 4 Dec — Handover week

The project is handed over: the documentation is complete and findable, the demo is rehearsed and the self-assessment written. The game is not touched — the work is made visible.

- ☐ Finish the project documentation so every claim has an exact reference.
- ☐ Rehearse the 8–10 minute demo and time it with a game team member.
- ☐ Write the self-assessment against the meeting notes and hand over the MVP on Fri 4 Dec.

Done when: Every claim in the documentation has at least one exact reference (commit, test id, note, release); the demo stays inside its time and covers the technique, a bug fix and the Git history; the self-assessment references real meeting notes.

In the Git repository before you tick the box: In the repo: the finished project documentation, the collected journal and AI log, and the self-assessment in project-docs.

The final five days

Mon 30 Nov · Content freeze — v1.0 is on the release page, journal and AI log committed

Tue 1 Dec · Finish the project documentation and its references

Wed 2 Dec · Rehearse the demo (8–10 min) with a game team member

Thu 3 Dec · Buffer: review the material with another person + self-assessment

Fri 4 Dec · Handover: the MVP + the project documentation to the client

Remember: the tick boxes and fields on the site are stored in your browser only. They do not reach your instructor and they do not replace what is in Git.